Comparative Analysis of Process Management in Linux and Windows

Assignment

Submitted By:

Rohit Sharma Pandey

M.E Computer 1st Semester

Submitted in Partial Fulfillment of the Requirement of Master's Degree in ME Computer

Submitted to:

Everest Engineering College

Sanepa, Lalitpur-2



September 2026

Abstract

Process management is a fundamental responsibility of an operating system, involving the creation, scheduling, execution, synchronization, communication, and termination of processes. Linux and Microsoft Windows implement these functions using different architectural approaches and kernel designs. This comparative study analyzes Linux and Windows process management with respect to process creation, scheduling approaches, process states, priority handling, and inter-process communication (IPC). Although both operating systems support multitasking and multiprocessing, Linux generally follows a UNIX-based model emphasizing flexible process creation and configurable scheduling, whereas Windows uses a thread-centric architecture with priority-driven scheduling and kernel-managed synchronization mechanisms.

I. INTRODUCTION

A process is a program in execution together with its associated resources, execution context, memory space, and operating system information. Efficient process management is essential for system responsiveness, fairness, resource utilization, and reliability. Linux and Windows are widely used operating systems with significantly different internal architectures. This study compares their process management mechanisms based on process creation, scheduling approach, process states, priority handling, and inter-process communication.

II. COMPARISON TABLE

Aspect	Linux	Windows
Process Creation	Primarily uses fork() to create a new process and exec() to load a new program. Copy-on-write improves efficiency.	Commonly uses CreateProcess(), which creates and initializes a new process and its primary thread.
Scheduling Approach	Uses multiple scheduling classes and policies. General-purpose workloads are traditionally handled by CFS, with real-time policies also available.	Uses a preemptive, priority-driven scheduler. Threads are the primary schedulable entities.
Process States	Includes running/runnable, interruptible sleep, uninterruptible sleep, stopped/traced, and terminated/zombie states.	Uses thread-oriented states such as ready, running, waiting, terminated, and transition-related states.
Priority Handling	Uses nice values for normal workloads and separate real-time scheduling priorities and policies.	Uses process priority classes and thread priority levels, with dynamic priority adjustments.
Inter-process Communication	Supports pipes, FIFOs, signals, message queues, shared memory, sockets, and semaphores.	Supports anonymous and named pipes, shared memory/file mapping, RPC, sockets, mailslots, and synchronization objects.

III. DETAILED COMPARATIVE ANALYSIS

A. Process Creation

Linux traditionally follows the UNIX process creation model. A new process is commonly created using the fork() system call. The child initially receives a logical copy of the parent process environment, while copy-on-write allows memory to be shared until modifications occur [1]. A process may then execute a different program using the exec() family of functions. Linux also provides lower-level mechanisms such as clone(), which permit selective resource sharing and support thread implementation.

Windows commonly creates processes using the CreateProcess() API. This mechanism establishes a new process, address space, and initial thread in an integrated operation [2]. Unlike the traditional fork-and-exec model, Windows typically combines process creation and program initialization through a unified API.

Comparison: Linux separates process duplication from program execution, which gives more flexibility, while Windows combines creation and initialization into a single, integrated mechanism.

B. Scheduling Approach

Linux supports multiple scheduling classes for different workloads. General-purpose tasks are traditionally managed using the Completely Fair Scheduler (CFS), whose objective is to distribute processor time fairly among eligible tasks [3]. Linux also supports real-time policies such as SCHED_FIFO and SCHED_RR.

Windows uses a preemptive, priority-driven scheduler. The scheduler generally selects the highest-priority ready thread for execution. Threads, rather than processes, are the fundamental scheduling entities [2]. Windows may dynamically adjust priorities to improve responsiveness.

Comparison: Linux focuses on fairness through configurable scheduling policies, while Windows focuses on priority-driven execution with dynamic responsiveness.

C. Process States

Linux commonly represents processes using states such as running/runnable, interruptible sleep, uninterruptible sleep, stopped, traced, and zombie. The zombie state allows a parent process to obtain termination information from a child process [1].

Windows is more strongly thread-oriented and commonly uses conceptual states such as ready, running, waiting, terminated, and transition-related states. Since threads are the primary execution units, thread state is particularly important in Windows scheduling [2].

Comparison: Linux keeps distinctive UNIX lifecycle states such as zombie and uninterruptible sleep, while Windows centers its process lifecycle on individual threads rather than processes.

D. Priority Handling

Linux supports priority control through nice values and scheduling policies. For ordinary processes, nice values traditionally range from -20 (higher priority) to +19 (lower priority). Real-time workloads use separate scheduling classes and priorities [3].

Windows uses process priority classes together with thread priority levels. The system may dynamically boost thread priorities in selected situations to improve interactive performance and responsiveness [2].

Comparison: Linux combines priority values with scheduling policies, while Windows relies on a thread-centric priority framework with dynamic adjustments.

E. Inter-Process Communication (IPC)

Linux provides numerous IPC mechanisms, including anonymous pipes, named pipes (FIFOs), signals, message queues, shared memory, semaphores, UNIX domain sockets, and network sockets. Shared memory can provide high-speed communication, while synchronization primitives coordinate access to shared resources [1].

Windows provides anonymous and named pipes, shared memory through file mapping, sockets, Remote Procedure Call (RPC), mailslots, and synchronization objects such as events, mutexes, and semaphores [2].

Comparison: Linux IPC follows UNIX and POSIX conventions, while Windows IPC is built around its object and handle architecture. Both systems support pipes, shared memory, sockets, and synchronization mechanisms.

IV. OVERALL COMPARATIVE DISCUSSION

Linux and Windows both offer advanced process management, but they follow different design philosophies. Linux takes a flexible, UNIX-oriented approach built on fork(), exec(), configurable scheduling policies, and a range of IPC mechanisms. Windows takes a more integrated, object-based and thread-centric approach, with priority-driven scheduling and extensive kernel synchronization facilities. Linux suits flexible systems programming and server workloads well, while Windows offers strong support for interactive desktop and enterprise environments.

V. CONCLUSION

Both Linux and Windows provide advanced mechanisms for efficient process management. Linux uses a flexible UNIX-based model centered on fork(), exec(), configurable scheduling policies, and extensive IPC facilities. Windows uses an integrated process creation mechanism and a priority-driven, thread-oriented scheduling architecture. Even with these architectural differences, both operating systems meet the same core objectives: multitasking, efficient CPU utilization, synchronization, communication, and controlled execution of multiple programs. Which approach is preferable depends on the workload, the application, and the system's specific objectives.

REFERENCES

[1] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook. San Francisco, CA, USA: No Starch Press, 2010.

[2] M. Russinovich, D. A. Solomon, and A. Ionescu, Windows Internals, 7th ed. Redmond, WA, USA: Microsoft Press, 2017.

[3] Linux Kernel Documentation, “Completely Fair Scheduler (CFS) Design,” Linux Kernel Documentation.

[4] Microsoft, “Processes and Threads,” Microsoft Windows Documentation.

[5] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.